Three consts, three meanings. Read them right to left.

```
const int* a;
int* const b;
const int* const c;
```

They look almost the same. They are three different rules.

swipe →

The rule

Start at the name. Read **right to left**.

DECLARATION	READ AS
`const int* a`	pointer to a **const int**
`int* const b`	**const pointer** to an int
`const int* const c`	const pointer to a const int

The `*` splits the line: `const` on its left locks the value, on its right locks the pointer.

What you can change

	POINT ELSEWHERE	CHANGE THE INT
`const int* a`	yes	no
`int* const b`	no	yes
`const int* const c`	no	no

Each of the six cells was checked with the compiler.

The code

```
int x = 1, y = 2;

const int* a = &x;
a = &y;         // ok
*a = 5;         // error

int* const b = &x;
b = &y;         // error
*b = 5;         // ok
```

`const int*` and `int const*` are the same type. Only the position of `const` *after* the `*` changes the meaning.

Where you meet it

```
size_t length(const char* s);


int* p = &x;
const int* q = p;   // ok

const int* r = &x;
int* t = r;         // error
```

You can add `const`, never silently remove it.

That is why `const char*` parameters accept both: the function promises not to write.

Remember this

CONST IS	IT LOCKS
left of the `*`	**the value**
right of the `*`	**the pointer**

Which do you write: `const int*` or `int const*`?